

SYNTAX HIGHLIGHTING

Shell blocks that read as code

A command list is the shape a shell block actually takes on a slide. Stock highlight.js gives it one token in six lines. Lattice ships its own grammar rather than borrowing a wrong one.

The commands a deck actually shows now carry color.

bash · package managers, container and cloud CLIs, flags

```
npm install @workwel/lattice
npx lattice build deck.md --theme indaco
docker compose up -d --build
kubectl apply -f k8s/ --namespace prod
terraform apply -auto-approve
```

SAME SOURCE • TWO GRAMMARS

Painting more is not the same as painting correctly.

POWERSHELL ON BASH • LIVELY AND WRONG

```
OUT_DIR=${OUT_DIR:-dist}  
git tag -m "release 1.4.0"  
cp my-file.txt "$OUT_DIR"
```

LATTICE SHELL • LIVELY AND RIGHT

```
OUT_DIR=${OUT_DIR:-dist}  
git tag -m "release 1.4.0"  
cp my-file.txt "$OUT_DIR"
```

What the borrowed grammar gets wrong is invisible at a glance

SHELL HIGHLIGHTING · OWNED GRAMMAR

- A default value read as a flag
 - In `${OUT_DIR:-dist}`, `-dist` is the fallback value, not an option.
- Strings that stop being strings
 - A quoted commit message renders as bare words; `v1.4.0` becomes the number `1.4`.
- It fails in the direction that survives review
 - The slide looks more alive, not less. Nobody catches it in the room.

A real script keeps everything stock bash got right, and gains the rest.

bash · shebang, control flow, substitution

```
#!/usr/bin/env bash
set -euo pipefail

for f in "$OUT_DIR"/*.css; do
  if [[ ! -s "$f" ]]; then
    echo "empty: $f" >&2
    exit 1
  fi
done
```

TAG THE SHAPE • TWO GRAMMARS

`sh` is a script; `console` is a transcript. The lint knows.

SH • THE SCRIPT GRAMMAR

```
kubectl apply -f k8s/  
helm upgrade api ./chart
```

CONSOLE • THE PROMPT IS A PROMPT

```
$ npm run build  
built 19 slides
```

Two tags, two grammars, one message

SHELL HIGHLIGHTING · OWNED GRAMMAR

- `sh`, `bash`, `zsh` parse a script
 - Control flow, substitution and quoting — with commands and flags on top.
- `shell`, `console` mark a transcript
 - Their job is the `$` prompt, so a script tagged that way colors almost nothing.
- The linter catches the mix-up
 - It names both tags and the fix, so nobody guesses why a block looks flat.